

PRAKTIKUM SISTEM OPERASI

MODUL 5



Oleh:

(Resita Istantia Purwanto)

(2311104037)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

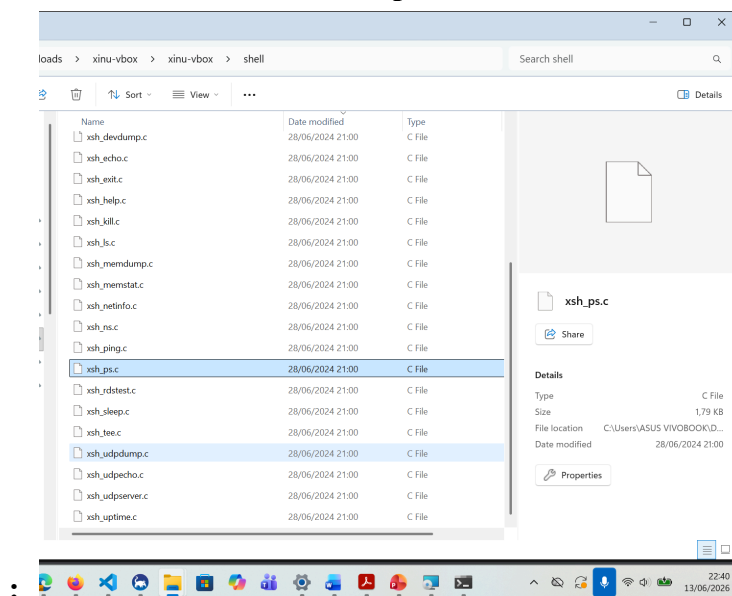
2026

JURNAL/TP

1. Jawablah pertanyaan berikut ini:

- a. Berapa banyaknya maksimum proses yang ada pada Xinu?
: Maksimum proses ditentukan oleh konstanta NPROC. Biasanya 50 proses (tergantung konfigurasi, tapi default Xinu = 50)
- b. Berapa maksimal panjang nama suatu proses pada Xinu?
: Ditentukan oleh PNMLEN Biasanya 16 karakter (termasuk null terminator)
- c. Berapa nilai prioritas awal pada saat proses dibuat?
; Ditentukan saat pemanggilan `create()` Default umumnya 20
- d. Ada berapa total state pada Xinu? Sebutkan!
: 6 state utama, yaitu:
 - PR_FREE → slot proses kosong
 - PR_CURR → proses sedang berjalan
 - PR_READY → siap dijalankan
 - PR_RECV → menunggu pesan
 - PR_SLEEP → sedang tidur (delay)
 - PR_SUSP → di-suspend

2. Perintah `ps` adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari `ps` tersimpan pada file `xsh_ps.c`. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!



```
on View Go ... PRAKTIKUM PPL
style inventory.py transaction.py item.json report.py utility module.py locustfile.py math_operation.py vbmangrotoppy C process.h C shell.h C xsh_psc x ...
C:\Users> ASUS VIVOROCK > Downloads > amu-dbox > amu-dbox > shell > C xsh_psc
1 /* xsh_psc.c - xsh_ps */
2
3 #include <xm.h>
4 #include <stdio.h>
5 #include <string.h>
6
7 /*-----*/
8 /* xsh_ps - shell command to print the process table
9 *-----*/
10
11 shellcmd xsh_ps(int nargs, char *args[])
12 {
13     struct proctent *prptr; /* Pointer to process */
14     int32 i; /* Index into proctabl */
15     char *pstate[] = { /* Names for process states */
16         "free", "curr", "ready", "exec", "sleep", "hang",
17         "wait", "rtime" };
18
19     /* For argument '--help', exit help about the 'ps' command */
20
21     if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
22         printf("Use: %s\n", args[0]);
23         printf("Description:\n");
24         printf("\tDisplays information about running processes\n");
25         printf("Options:\n");
26         printf("\t--help\t display this help and exit\n");
27         return 0;
28     }
29
30     /* Check for valid number of arguments */
31
32     if (nargs > 1) {
33         fprintf(stderr, "%s: too many arguments\n", args[0]);
34         fprintf(stderr, "Try %s --help for more information\n",
35             args[0]);
36         return 1;
37     }
38
39     /* MODIFIKASI: Menambahkan kolom Msg dan Content pada Header */
40     /*-----*/
41     printf("%3s %16s %5s %4s %3s %10s %4s %10s %10s\n",
42         "Pid", "Name", "State", "Prior", "Msg", "Content", "Ppid", "Stack Base",
43         "Stack Ptr", "Stack Size");
44
45     printf("%3s %16s %5s %4s %3s %10s %4s %10s %10s\n",
46         "-----", "-----", "-----", "-----", "-----", "-----", "-----", "-----",
47         "-----", "-----");
48
49     /* Output information for each process */
50
51     for (i = 0; i < NPROC; i++) {
52         prptr = &proctabl[i];
53         if (prptr->pstate == PR_FREE) { /* Skip unused slots */
54             continue;
55         }
56
57         /* MODIFIKASI: Logika untuk mengacak pesan di dalam buffer proses */
58         /*-----*/
59         /* MODIFIKASI: Format cetak data disesuaikan dengan kolom baru */
60         printf("%3d %16s %5s %4s %3d %10d %4d %10d %10d\n",
61             i, prptr->prname, pstate[i], prptr->pstate,
62             prptr->pprio, msg_count, msg_content, prptr->pparent,
63             prptr->prstibase, prptr->prstiptr, prptr->prstklen);
64     }
65     return 0;
66 }
```

```
tion View Go ... PRAKTIKUM PPL
style inventory.py transaction.py item.json report.py utility module.py locustfile.py math_operation.py vbmangrotoppy C process.h C shell.h C xsh_psc x ...
C:\Users> ASUS VIVOROCK > Downloads > amu-dbox > amu-dbox > shell > C xsh_psc
12
32     if (nargs > 1) {
33         fprintf(stderr, "Try %s --help for more information\n",
34             args[0]);
35         return 1;
36     }
37
38     /* MODIFIKASI: Menambahkan kolom Msg dan Content pada Header */
39     /*-----*/
40     printf("%3s %16s %5s %4s %3s %10s %4s %10s %10s\n",
41         "Pid", "Name", "State", "Prior", "Msg", "Content", "Ppid", "Stack Base",
42         "Stack Ptr", "Stack Size");
43
44     printf("%3s %16s %5s %4s %3s %10s %4s %10s %10s\n",
45         "-----", "-----", "-----", "-----", "-----", "-----", "-----", "-----",
46         "-----", "-----");
47
48     /* Output information for each process */
49
50     for (i = 0; i < NPROC; i++) {
51         prptr = &proctabl[i];
52         if (prptr->pstate == PR_FREE) { /* Skip unused slots */
53             continue;
54         }
55
56         /* MODIFIKASI: Logika untuk mengacak pesan di dalam buffer proses */
57         /*-----*/
58         /* MODIFIKASI: Format cetak data disesuaikan dengan kolom baru */
59         printf("%3d %16s %5s %4s %3d %10d %4d %10d %10d\n",
60             i, prptr->prname, pstate[i], prptr->pstate,
61             prptr->pprio, msg_count, msg_content, prptr->pparent,
62             prptr->prstibase, prptr->prstiptr, prptr->prstklen);
63     }
64     return 0;
65 }
```

3. Ubahlah perintah ps (source code: xsh_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini Kolom Msg adalah banyaknya pesan yang ada dalam proses. Kolom Content adalah isi dari pesan tersebut.

Langkah pengerjaan:

- **Modifikasi source code pada file xsh_ps.c**
 - **Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya**
 - **Jalankan Backend VM**
 - **Setelah sistem berjalan, jalankan perintah \$ps. Pastikan hasilnya sesuai dengan contoh output pada gambar yang diberikan.**
- Screenshot source kode dan output akhir hasil modifikasi**

- 4. [35 Poin]** Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu sejak booting **hanya dalam satuan menit.**

Langkah pengerjaan:

- **Modifikasi source code pada file xsh_uptime.c**
- **Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya**
- **Jalankan Backend VM**
- **Setelah sistem berjalan, jalankan perintah \$uptime. Pastikan hasilnya sesuai dengan contoh output yang diinginkan**
- **Screenshot source kode dan output akhir hasil modifikasi**